

Advanced Data Management

Lab 4 — Operators: joins, sorting, and aggregation

Cycle Ingénieur · S3

Fall 2026

Goals

Read execution plans operator by operator; compare the three join algorithms on the same query; trigger an external sort by shrinking memory; and see hash vs. sort aggregation. Uses `adm_lab` (grow `orders` to ~500k rows as in Lab 3 for clearer effects).

Note. Use `SET` to *force* algorithms for teaching, and `RESET ALL`; between parts so a leftover setting does not distort the next plan.

1. Read a plan

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT c.name, SUM(o.amount) AS total
FROM customers c JOIN orders o ON o.cust_id = c.id
WHERE c.country = 'MA'
GROUP BY c.name ORDER BY c.name;
```

List every operator, bottom-up, and mark the pipeline breaker.

2. Force the three joins

```
EXPLAIN ANALYZE SELECT * FROM customers c JOIN orders o ON o.cust_id = c
.id;    -- default
SET enable_hashjoin = off;
EXPLAIN ANALYZE SELECT * FROM customers c JOIN orders o ON o.cust_id = c
.id;    -- merge/nestloop
SET enable_mergejoin = off;
EXPLAIN ANALYZE SELECT * FROM customers c JOIN orders o ON o.cust_id = c
.id;    -- nested-loop
RESET ALL;
```

Record the algorithm and total time for each of the three runs.

3. External sort

```
SET work_mem = '64kB';
EXPLAIN ANALYZE SELECT * FROM orders ORDER BY amount;
SET work_mem = '256MB';
EXPLAIN ANALYZE SELECT * FROM orders ORDER BY amount;
RESET work_mem;
```

Copy the `Sort Method` line for both settings.

4. Hash vs. sort aggregation

```
EXPLAIN ANALYZE SELECT country, count(*) FROM customers GROUP BY country
;
EXPLAIN ANALYZE SELECT name, count(*) FROM customers GROUP BY name;
```

5. Size flips the join

```
EXPLAIN ANALYZE
  SELECT * FROM customers c JOIN orders o ON o.cust_id = c.id WHERE c.id
    < 10;
EXPLAIN ANALYZE
  SELECT * FROM customers c JOIN orders o ON o.cust_id = c.id;
```

6. Exercises to submit

1. Take the query from Part 1. From its `EXPLAIN (ANALYZE, BUFFERS)` output, label each operator and identify the pipeline breaker.
2. Force nested-loop, merge, and hash join on the same query. Report the three times and say which won and *why*, in terms of sizes and order.
3. Trigger an external sort by lowering `work_mem`. Paste the `Sort Method` line for the small and the large setting, and explain the change.
4. Find one `GROUP BY` that the planner runs as `HashAggregate` and one it runs via sorting. Explain the difference in terms of the number of groups.
5. Paper exercise (from the lecture): for a block nested-loop join with $B_R = 200$, $B_S = 2000$, and $M = 22$ buffer pages, compute the approximate page-read cost. Show your work. (*Hint: $B_R + \lceil B_R / (M - 2) \rceil \cdot B_S$.*)

Deliverable

A Jupyter notebook `adm_lab4_<yourname>.ipynb` with the plans, the three join timings, the two `Sort Method` lines, the aggregation comparison, and your worked answer to the paper exercise. Attach the `.sql` you used.

Troubleshooting

- A forced setting seems ignored: some plans have no alternative — try a larger/smaller query, and confirm you `RESET` between parts.
- No external sort appears: lower `work_mem` further, or sort a larger result set.